K L (Deemed to be) University

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING

PROJECT REPORT

Data Structures and Algorithms - 3 (25CS2103E)

VERITEXT

Plagiarism Detection Engine for Academic Submissions

Academic Year: 2026–2027

Submitted by

V. Hari Priya – 2520080015

M. Sai Harika – 2520080068

Anirudh Ganesh- 2520080058

Team Number: 1 Section: 13

Under the guidance of

Guide Name / Designation / Department

Hyderabad – 500090, Telangana, India.

DECLARATION

We hereby declare that the project report entitled “VeriText – Plagiarism Detection Engine for Academic Submissions” is a record of our bonafide work submitted in partial fulfilment of the requirements for the Data Structures and Algorithms – 3 course at K L (Deemed to be) University during the academic year 2026–2027.

The project was developed as a team-based implementation of data structures and algorithms for detecting similarity between academic submissions. The work presented in this report is our own and has not been copied from any other department, university, institute, or previously submitted project.

CERTIFICATE

This is to certify that the mini project report entitled “VeriText – Plagiarism Detection Engine for Academic Submissions” is a bonafide work carried out and submitted by the students named below in partial fulfilment of the requirements for the Data Structures and Algorithms – 3 course at K L (Deemed to be) University during the academic year 2026–2027.

V.HariPriya – 2520080015

M.Sai Harika – 2520080068

Anirudh Ganesh – 2520080058

Signature of the Supervisor

ACKNOWLEDGEMENT

We express our sincere gratitude to our project guide and faculty members for their guidance, encouragement, and valuable suggestions throughout the development of VeriText. Their support helped us understand the practical application of data structures and algorithms in a real-world academic problem.

We are grateful to the Department of Computer Science Engineering, K L (Deemed to be) University, for providing the facilities and learning environment required to complete this project.

We also thank our classmates, friends, and family members for their cooperation and support during development, testing, and documentation.

TABLE OF CONTENTS

1. Abstract

2. Introduction

3. Literature Survey

4. Problem Statement and Objectives

5. Hardware and Software Requirements

6. Implementation

7. Results & Discussion

8. Conclusion & Future Scope

9. References

1. ABSTRACT

VeriText is a web-based plagiarism detection and academic evaluation system designed to help faculty identify similarities between student submissions in an organized and efficient manner. The system follows a faculty-centered workflow in which students upload assignments, while faculty members review submissions, perform plagiarism analysis, enter marks and feedback, and publish the final report to the student.

The core plagiarism engine applies string-processing and data-structure concepts to compare submissions. Rabin–Karp fingerprinting is used to generate hashes for text windows, while winnowing reduces the number of fingerprints considered for comparison. Candidate similarities are further examined using suffix-array and Longest Common Prefix (LCP) concepts to identify meaningful common passages. The system records similarity percentage, fingerprint statistics, common fingerprints, and matching passages as part of the evaluation.

The application is implemented using a FastAPI backend, SQLite with SQLAlchemy, Python processing modules, and an HTML/CSS/JavaScript frontend. A dynamic faculty roster allows students who have not submitted an assignment to be identified, and Excel export provides class-level tracking. VeriText demonstrates how DSA concepts can be integrated into a practical academic software system.

2. INTRODUCTION

Academic institutions receive many digital assignments from students. Manually checking every submission for duplicated or highly similar content is time-consuming and can lead to inconsistent evaluation. A software-based plagiarism detection system can assist faculty by identifying potentially similar passages and presenting the evidence in a structured report.

VeriText addresses this problem using Data Structures and Algorithms. The system processes submitted text into searchable fingerprints and compares those fingerprints between documents. This reduces unnecessary comparisons and allows the system to identify longer common passages.

The application separates student and faculty responsibilities. Students submit assignments and view published reports, while faculty members inspect submissions, run similarity analysis, enter marks and feedback, and publish results.

The report describes the algorithms, system modules, requirements, implementation, testing observations, results, and future enhancements.

3. LITERATURE SURVEY

Turnitin:

An academic integrity platform that compares submissions against large collections of academic and online content and provides similarity reports.

Grammarly Plagiarism Checker:

A writing-support tool that demonstrates user-oriented text comparison and plagiarism reporting.

Copyleaks:

A plagiarism and content-analysis platform demonstrating scalable document processing and similarity reporting.

Unicheck:

An academic plagiarism detection solution focused on document comparison and instructor-oriented reporting.

Quetext:

A plagiarism detection service that highlights matching content, providing a useful reference for evidence-based reports.

MOSS:

Measure Of Software Similarity, used for detecting similarities among source-code submissions and demonstrating specialized comparison techniques.

JPlag:

A source-code plagiarism detection system that compares program submissions and identifies structural similarities.

4. PROBLEM STATEMENT AND OBJECTIVES

Problem Statement: Develop a web-based system that efficiently identifies and reports textual similarity between academic submissions while supporting a faculty-driven evaluation workflow.

Problem Description: Manual comparison of multiple assignments is time-consuming and makes consistent identification of duplicated passages difficult. A structured algorithmic system is required to compare submissions, quantify similarity, identify common content, and maintain evaluation records.

Objectives:

- Provide a centralized platform for student assignment submission and faculty evaluation.
- Generate text fingerprints using Rabin–Karp hashing.
- Apply winnowing to reduce the fingerprint set while preserving representative matches.
- Use suffix-array and LCP concepts to identify meaningful common passages.
- Calculate similarity percentage with supporting fingerprint statistics.
- Allow faculty to combine plagiarism analysis with marks and feedback.
- Generate a final evaluation report for the student.
- Maintain a dynamic class roster including students who did not submit.
- Provide Excel export for class-level tracking.
- Demonstrate practical application of DSA concepts in an academic system.

5. HARDWARE AND SOFTWARE REQUIREMENTS

Hardware Requirements

- Laptop/Desktop computer with a modern multi-core processor.
- Minimum 4 GB RAM; 8 GB or more recommended.
- At least 1 GB free storage for application data and submissions.
- Keyboard, mouse/touchpad, and network connection for development.

Software Requirements

- Python 3.x
- FastAPI and Uvicorn
- SQLAlchemy and SQLite
- OpenPyXL
- HTML5, CSS3 and JavaScript
- Visual Studio Code
- Google Chrome or another modern browser

6. IMPLEMENTATION

VeriText is implemented as a full-stack web application containing frontend pages, a FastAPI backend, database models, document-processing components, a plagiarism engine, roster management, evaluation storage, and report pages.

Module 1 – Student Submission

Students can log in, submit assignments, view submission history, maintain profile information, and access published evaluation reports. Students do not run plagiarism analysis.

Module 2 – Faculty Evaluation and Roster

Faculty can view submissions, evaluate assignments, run plagiarism analysis, enter marks and feedback, publish reports, view class rosters, identify non-submitters, and export the roster to Excel.

Module 3 – Plagiarism Detection Engine

Rabin–Karp rolling hashing generates fingerprints. Winnowing selects representative fingerprints. Suffix-array and LCP-based comparison identifies longer common passages. The engine returns similarity percentage and supporting statistics.

Module 4 – Database and Reports

SQLite and SQLAlchemy store sections, students, submissions, and evaluations. Evaluation records contain marks, similarity percentage, fingerprint statistics, common passages, feedback, faculty name, and status.

Core DSA concepts used:

- Hashing / fingerprint indexing
- Rabin–Karp rolling hash
- Winnowing
- Suffix arrays
- Longest Common Prefix (LCP)
- Lists and structured records
- Sorting
- Database indexing and relational links

7. RESULTS & DISCUSSION

The developed system demonstrates an end-to-end academic plagiarism evaluation workflow. Students can submit assignments through the web interface, and faculty can retrieve submissions, perform similarity analysis, enter marks and feedback, and publish the final evaluation.

Engine testing produced similarity scores together with fingerprints, winnowed fingerprints, common fingerprints, and detected common passages. These values provide supporting evidence for the similarity score rather than presenting only a single percentage.

The database-backed roster distinguishes students who submitted assignments, students whose submissions are awaiting evaluation, evaluated students, and students who did not submit. Excel export provides a class-level record containing submission and evaluation information.

Recommended screenshots to insert:

- VeriText home page
- Student login/dashboard
- Student assignment submission
- Faculty dashboard
- Faculty evaluation page with analysis results
- Faculty roster showing submitted/non-submitted students
- Faculty detailed report
- Student final evaluation report

8. CONCLUSION & FUTURE SCOPE

VeriText successfully implements a web-based plagiarism detection and academic evaluation workflow using Data Structures and Algorithms. The system provides separate student and faculty interfaces, assignment submission, dynamic class-roster management, algorithmic similarity analysis, evaluation storage, and final reporting.

The project demonstrates practical use of hashing, Rabin–Karp fingerprinting, winnowing, suffix arrays, LCP-based substring comparison, sorting, lists, and database-backed records. By combining these concepts with FastAPI and a browser-based interface, VeriText converts DSA techniques into a usable academic application.

Future Scope

- Support PDF and DOCX text extraction.
- Add phrase and sentence comparison improvements.
- Introduce Trie-based indexing.
- Improve false-positive analysis using a larger labelled dataset.
- Add configurable similarity thresholds.
- Provide richer side-by-side passage highlighting.
- Add secure database-backed authentication and role authorization.
- Support institution-wide analytics and multiple faculty sections.
- Generate downloadable PDF evaluation reports.
- Optimize large-scale comparison using indexed fingerprints and parallel processing.

9. REFERENCES

FastAPI Documentation – https://fastapi.tiangolo.com/

Python Documentation – https://docs.python.org/3/

SQLAlchemy Documentation – https://www.sqlalchemy.org/

OpenPyXL Documentation – https://openpyxl.readthedocs.io/

Turnitin – https://www.turnitin.com/

Copyleaks – https://copyleaks.com/

Unicheck – https://unicheck.com/

Quetext – https://www.quetext.com/

MOSS – https://theory.stanford.edu/~aiken/moss/

JPlag – https://github.com/jplag/JPlag